

# AgroSearch: motor de busca textual com índice invertido e TF-IDF

Ricardo Silva Flores · Ciência da Computação · UNIPÊ

Tópicos Avançados (Recuperação de Informação / PLN) · Laboratório Prático 04 · Prof. Me. Ricardo Roberto de Lima

## 1. Objetivo

O AgroSearch é um protótipo, em um único arquivo Streamlit (**agrosearch.py**), que recebe a consulta de um técnico de campo e devolve os trechos dos manuais da AgroTech Solutions ordenados por relevância. Conforme a restrição do desafio, nenhuma biblioteca de alto nível foi usada: o pré-processamento, o índice invertido, o TF, o IDF, o TF-IDF e a similaridade de cosseno foram escritos do zero com a biblioteca padrão do Python (*string*, *unicodedata*, *math*). O pandas aparece apenas para montar as tabelas exibidas na tela.

## 2. Como o sistema foi construído

| Fase              | Implementação                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pré-processamento | Quatro funções encadeadas por <b>pipeline()</b> : (1) tokenização, trocando a pontuação por espaços e separando as palavras; (2) normalização, com caixa baixa e remoção de acentos pela decomposição Unicode NFKD; (3) remoção de stopwords a partir de uma lista própria em português, já sem acentos; (4) stemming com um stemmer próprio em etapas, inspirado no RSLP: redução de plural (ex.: -ões → -ão, -ais → -al), sufixos nominais (-amento, -ação, -idade, -ador...), sufixos verbais (-ar, -er, -ando...) aplicados só quando não houve sufixo nominal, e retirada da vogal final. Cada regra respeita um tamanho mínimo de radical. Ex.: irrigação e irrigar → <b>irrig</b> ; lagarta e lagartas → <b>lagart</b> . |
| Índice invertido  | A função <b>indexar()</b> percorre os tokens já limpos e gera o dicionário termo → [IDs dos documentos], ordenado. Na mesma passada guarda $f(t,d)$ , a frequência de cada termo em cada documento, que é reaproveitada no cálculo do TF. Exibido como tabela (com $df$ e IDF) ou via <code>st.json</code> .                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Busca e TF-IDF    | O índice define os documentos candidatos (os que contêm ao menos um termo da consulta) e cada um recebe o score acumulado dos termos distintos da consulta. A tabela é ordenada do maior para o menor score e a linha do vencedor fica destacada em verde, com o trecho em evidência.                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Bônus: cosseno    | Consulta e documento viram vetores TF-IDF esparsos (dicionários) e é calculado o cosseno entre eles. Pela normalização dos vetores, documentos longos não levam vantagem só por terem mais palavras. Um seletor na barra lateral permite ordenar por score ou por cosseno.                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Interface         | Barra lateral com checkboxes de stopwords e stemming e o critério de ordenação; três abas, uma por fase. A aba 1 mostra cada documento em todas as etapas do pipeline e o vocabulário atual; a aba 3 mostra os termos processados da consulta, avisa sobre termos fora do vocabulário e detalha TF, IDF e TF×IDF por termo.                                                                                                                                                                                                                                                                                                                                                                                                     |

$$\text{Fórmulas: } TF(t,d) = f(t,d) / |d| \quad \cdot \quad IDF(t) = \log_{10}(N / df(t)) \quad \cdot \quad \text{Score}(d) = \sum_{t \in q} TF(t,d) \times IDF(t) \\ \cos(q,d) = (q \cdot d) / (|q| \times |d|), \text{ com vetores TF-IDF da consulta } (q) \text{ e do documento } (d)$$

## 3. Testes realizados

Base com os 5 documentos do enunciado ( $N = 5$ ), com stopwords e stemming ativos:

| Consulta                    | Termos processados | Ranking (score TF-IDF)                           | Cosseno do 1º |
|-----------------------------|--------------------|--------------------------------------------------|---------------|
| lagartas na soja            | lagart, soj        | Doc 2 (0,1033) > Doc 4 (0,0775) > Doc 1 (0,0277) | 0,3099        |
| adubação verde para o milho | adub, verd, milh   | Doc 3 (0,2996)                                   | 0,6547        |
| irrigar com gotejamento     | irrig, gotej       | Doc 5 (0,1567) > Doc 1 (0,0497)                  | 0,4576        |
| cultivo orgânico            | cultiv, organ      | Doc 5 (0,1997)                                   | 0,5624        |

**Conferência manual** (consulta “irrigação da soja”, Doc 1 com  $|d| = 8$  termos): irrig aparece nos Docs 1 e 5, então  $IDF = \log_{10}(5/2) = 0,3979$ ; soj aparece nos Docs 1, 2 e 4, então  $IDF = \log_{10}(5/3) = 0,2218$ . Os dois termos têm  $TF = 1/8 = 0,125$  no Doc 1, logo  $\text{Score} = 0,125 \times 0,3979 + 0,125 \times 0,2218 = 0,0497 + 0,0277 = \mathbf{0,0775}$ , o mesmo valor mostrado pelo sistema. O Doc 1 vence porque é o único com os dois termos.

## 4. Análise do pré-processamento

Sem nenhum filtro, a base tem 66 tokens e 47 termos distintos. A remoção de stopwords reduz para 36 tokens e 32 termos, eliminando palavras como “a”, “de” e “para”, que apareceriam em quase todos os documentos e só gerariam ruído. Nesta base pequena o stemming não reduz o vocabulário, porque nenhum radical se repete com formas diferentes entre os documentos; o ganho aparece na consulta. A busca por “lagarta” (singular) não encontra nada sem stemming, já que o texto traz “lagartas”; com stemming, as duas formas viram lagart e os Docs

2 e 4 são retornados. O mesmo ocorre com “irrigar” em relação a “irrigação”. Vale notar também que, com o IDF clássico, um termo presente nos 5 documentos teria  $IDF = 0$  e não influenciaria o ranking, o que é o comportamento esperado para termos pouco discriminativos.

## 5. Divisão de tarefas

O trabalho foi feito individualmente. As etapas foram organizadas assim:

| Etapa                    | Responsável          | Atividades                                                                                        |
|--------------------------|----------------------|---------------------------------------------------------------------------------------------------|
| 1. Pré-processamento     | Ricardo Silva Flores | Tokenização, normalização, lista de stopwords e regras do stemmer.                                |
| 2. Indexação             | Ricardo Silva Flores | Estrutura do índice invertido e das frequências $f(t,d)$ .                                        |
| 3. Ranqueamento          | Ricardo Silva Flores | Funções de TF, IDF, score acumulado e similaridade de cosseno (bônus).                            |
| 4. Interface e validação | Ricardo Silva Flores | Abas e controles no Streamlit, testes das consultas, conferência manual dos cálculos e relatório. |

**Execução:** `pip install streamlit pandas` → `streamlit run agrosearch.py`